

Jev implementation

Give repeated decisions a clear place in your workflow.

We connect Jev to a specific decision in your application, then define the routes, thresholds and fallback and test them against reviewed examples.

[Service overview](#)

fortitudo.agency



Your software needs to know what happens after a decision.

For repeated routing, classification or evaluation decisions that need defined outputs and fallback behavior.

Jev is for structured decisions. Text or media generation needs a different model.

Unstructured decisions

Your application needs a defined category or route, rather than another open-ended answer.

No rule for uncertain results

The workflow needs to distinguish a result it can use from one that needs another route or a review.

An integration without evaluation

The decision logic has not been compared with representative, reviewed examples.

A tested decision connected to the next action.

A decision schema and a set of realistic test examples

Thresholds, routing branches and fallback behavior

A Jev API or supported LangChain connection in your application or harness

Evaluation results and guidance for maintaining the integration

Why work with us.

Get a result the application can use

We define the decision schema and connect its output to the next step in your system.

Decide when to ask for review

Thresholds and fallback paths specify what happens when confidence is low or the provider is unavailable.

Check the decision on your examples

We compare results with reviewed inputs before you use the integration to control the workflow.

From the first conversation to a clear handover.

- | | | | |
|----|--|----|--|
| 01 | Choose the decision and define the possible results. | 02 | Prepare examples and review the expected decisions. |
| 03 | Connect Jev and implement thresholds and fallback paths. | 04 | Evaluate the integration and document the operating tradeoffs. |

First checkpoint: define the possible decisions and review example inputs before connecting Jev.

A working result, checked together.

We agree what success looks like before delivery, then review the work against it.

Responses are parsed into the agreed decision schema.

Reviewed examples exercise each route and uncertainty path.

Provider errors and low-confidence outcomes reach the defined fallback.

Turn context into an actionable decision.

Model and task routing

Classify an incoming task and use the result to select the next model, worker or workflow branch.

Classification and prioritization

Turn context into typed decisions your software can act on, such as request category, priority or whether a review is needed.

Evaluate the result. Define the fallback.

Agent evaluation

Apply defined criteria to agent output or traces. Compare the decisions with reviewed examples before using the results to control a workflow.

Thresholds and fallback

Define how confidence affects the next action and what happens when the result is uncertain or the provider is unavailable.

Start at the right scale.

Jev decision integration

One defined decision, its application connection and a representative evaluation set.

We agree the deliverables, responsibilities and review points in a written proposal.

A clear scope before the work begins.

What we need from you

Representative inputs, expected decisions, application access and a TypeSafe AI account with the required access.

What shapes the scope

Number of decisions · Harness or application integration · Test data and evaluation depth

Scope and responsibilities

Jev returns structured decisions rather than generated text or media. We measure suitability on your examples; vendor speed and cost claims are not a project guarantee.

Your written proposal confirms price, timing, responsibilities, review points and final inclusions.

Before you decide.

Is Jev another content generator?

No. TypeSafe AI's Jev returns typed decisions and probabilities. It complements the models handling open-ended reasoning or generation.

Can this work inside a LangChain harness?

Yes. We can scope the supported LangChain integration or a direct API connection, then test it against your decision and failure cases.

Let's define the decision.

Show us the repeated decision, sample inputs and what should happen next. We'll define the integration and the examples needed to evaluate it.

Start a conversation

See our work

hello@fortitudo.agency

fortitudo.agency/contact

